

# Web Hosting and DNS

Beginner-friendly notes with simple explanations, examples, diagrams, and practical concepts for Linux, Apache, and AWS.

## 1. Introduction to Web Hosting

### What is web hosting?

Web hosting is a service that allows us to store a website's files on a server and make the website accessible to people through the internet.

A website contains files such as HTML, CSS, JavaScript, images, and other resources. These files must be stored on a computer that can respond to requests from users.

That computer is called a web server when it serves web content.

In simple words: Web hosting means keeping your website on a server so that other people can open it using a browser.

### Example

Suppose we create a website for a company called ABC.

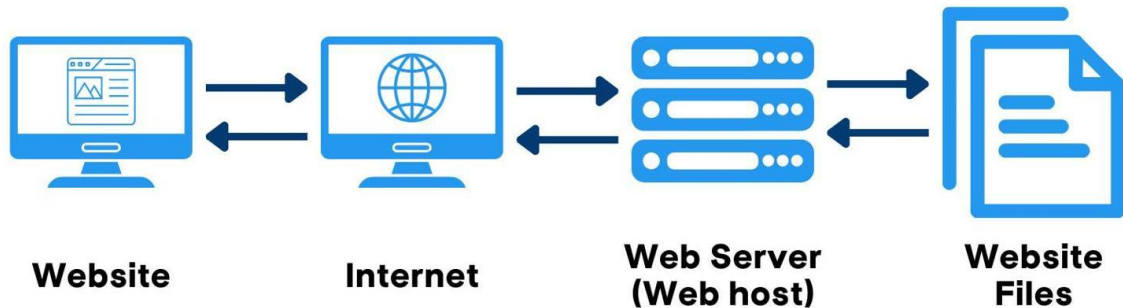
The website contains:

- `index.html` — the main webpage.
- `style.css` — controls the design and appearance.
- `script.js` — adds interactive features.
- `logo.png` — the company logo.

If these files are stored only on your laptop, other people generally cannot access the website over the internet.

If you upload these files to a web server and configure it properly, people can access the website through a domain name or IP address.

# How Web Hosting Works



## 2. What is a server?

A server is a computer or software system that provides services or resources to other computers, called clients.

For web hosting, the server stores or generates website content and sends it to users when they request it.

A server can be:

- A physical computer in a data center.
- A virtual machine running on a physical server.
- A cloud virtual machine, such as an AWS EC2 instance.

### Client and server

A client is the device or application requesting a service. A server provides the requested service.

For example:

| Client                | Server                       |
|-----------------------|------------------------------|
| Your laptop's browser | Web server hosting a website |
| Mobile browser        | Web server hosting a website |
| Email application     | Mail server                  |

When you open a website, your browser acts as the client, and the web server responds to the browser's request.

### What is a web server?

A web server can refer to the software that handles HTTP or HTTPS requests, or the computer running that software.

Common web server software includes:

- Apache HTTP Server (httpd)
- Nginx
- Microsoft IIS

In your Linux practical, you used Apache HTTP Server on RHEL 10.2.

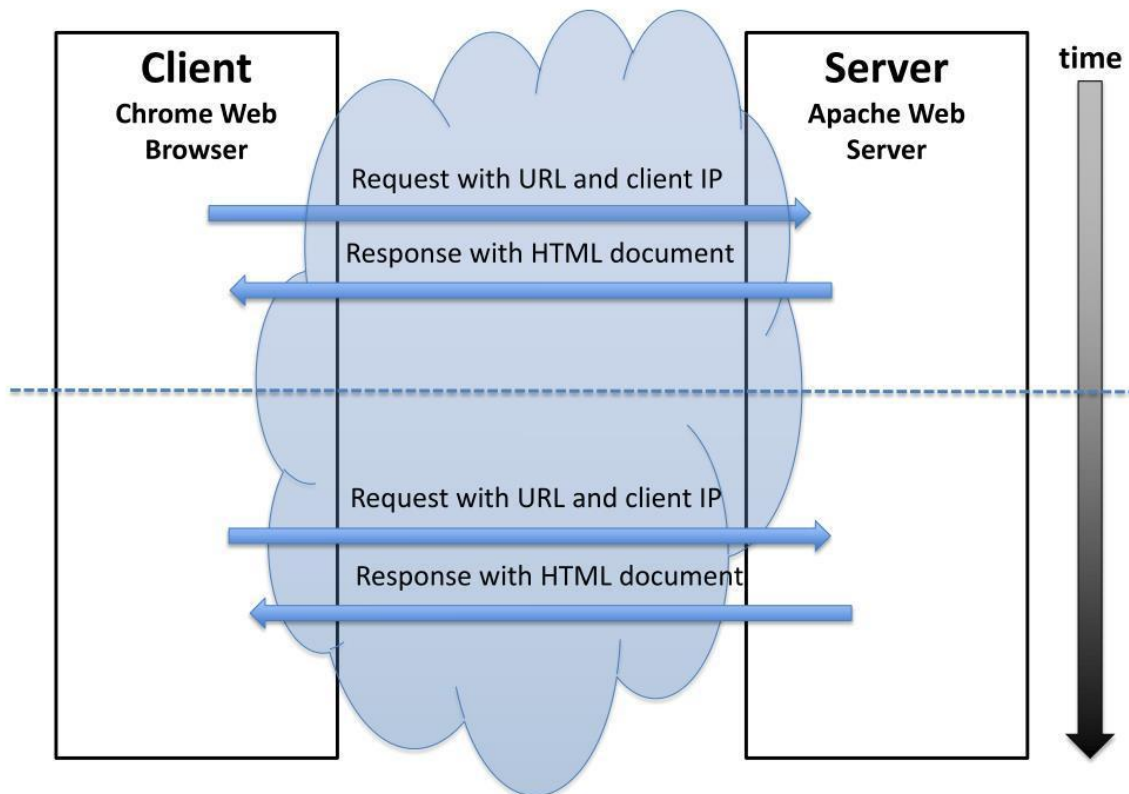
Apache receives HTTP requests and sends website content back to the client.

## 3. How web hosting works

Let's understand the complete process using a simple example.

Suppose a website is hosted on a server with the IP address `192.0.2.10`. This address is reserved for documentation and is only an example.

The website's domain name is [www.example.com](http://www.example.com).



## Step-by-step working

1. The user enters a website address.

The user opens a browser and enters [www.example.com](http://www.example.com).

2. The browser finds the server's IP address.

The browser uses DNS to find the IP address associated with the domain name.

3. The browser connects to the server.

It establishes a network connection to the server using the appropriate port and protocol.

4. The browser sends an HTTP request.

The request tells the web server which resource the browser wants, such as the homepage.

5. The web server processes the request.

Apache finds the requested file or passes the request to an application that generates the content.

6. The server sends a response.

Apache returns the requested content, such as HTML, along with an HTTP status code.

7. The browser displays the website.

The browser processes the HTML, CSS, JavaScript, and images to display the webpage.

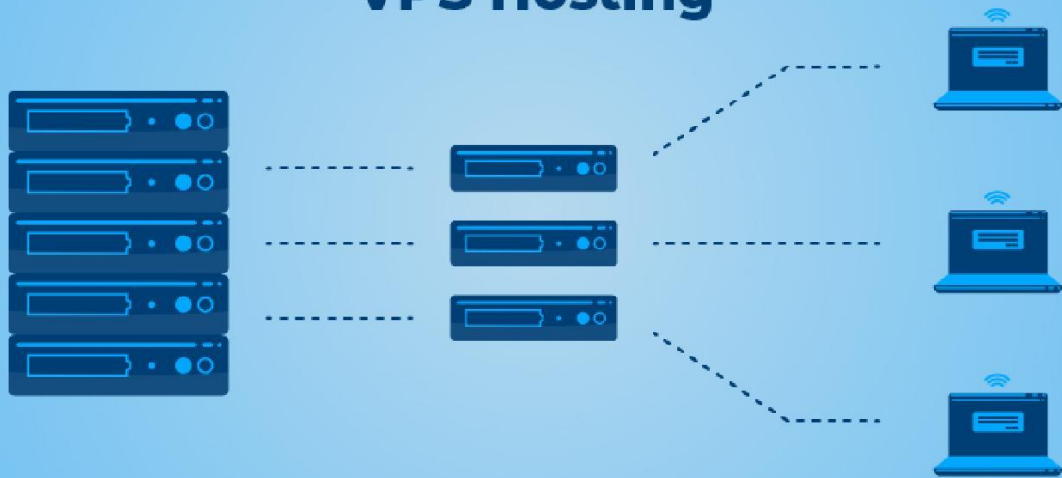
Important: DNS finds the server's address. Apache handles the web request. The browser displays the returned content.

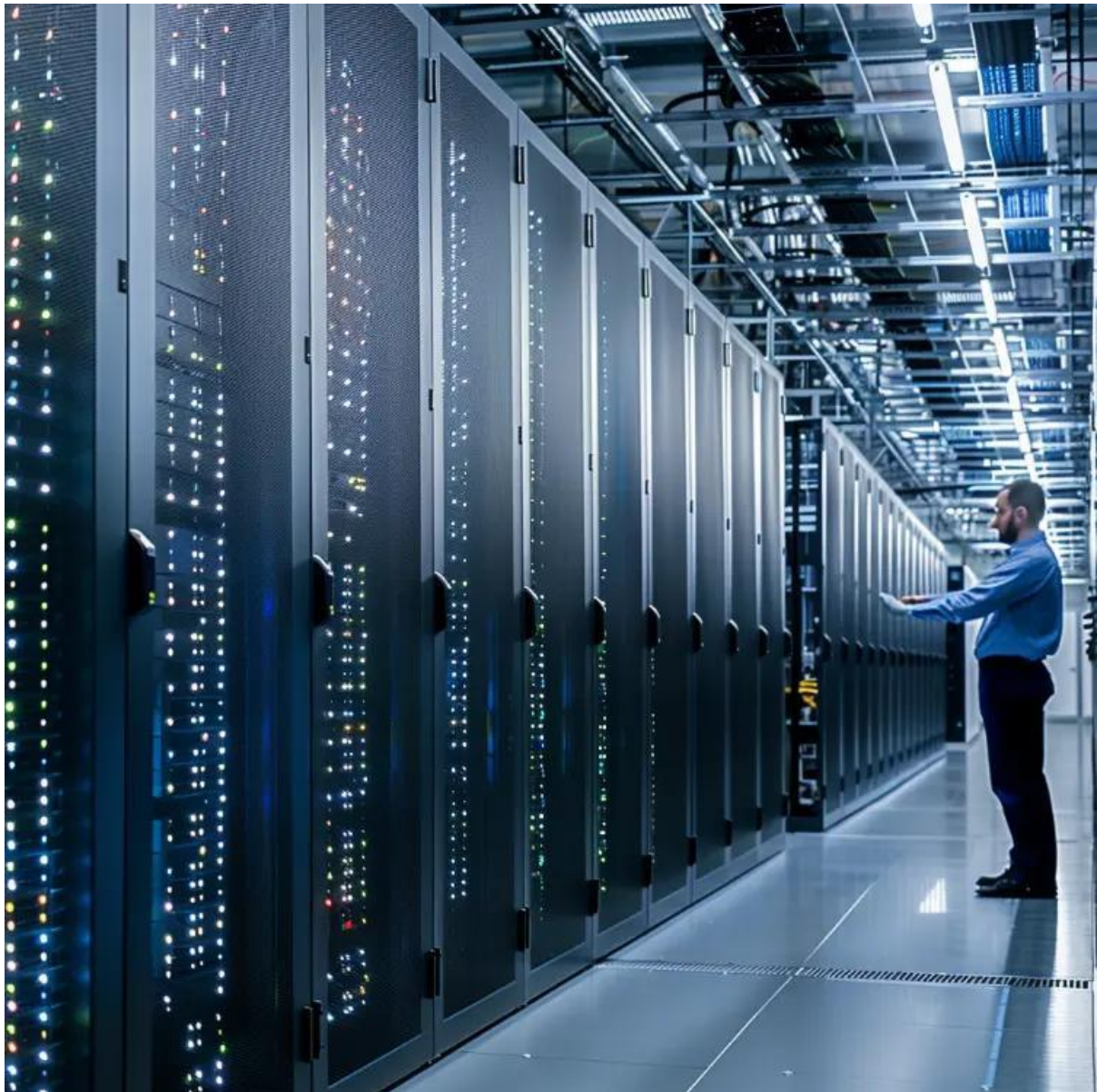
## 4. Types of web hosting

There are different ways to host a website depending on the requirements, traffic, cost, and level of control needed.



# VPS Hosting





## 4.1 Shared hosting

In shared hosting, multiple websites use the same physical server and share its resources.

For example, 100 websites may be hosted on one server.

Advantages:

- Low cost.
- Easy to manage.
- Suitable for small websites.

Disadvantages:

- Resources are shared.
- Limited control over server configuration.
- Heavy usage by one website can affect others.

## 4.2 VPS hosting

VPS stands for Virtual Private Server.

A physical server is divided into multiple virtual servers. Each VPS operates as an independent virtual machine with allocated resources.

Advantages:

- More control than shared hosting.
- Dedicated or allocated resources.
- Suitable for learning server administration.

Example: A Linux VPS running Apache to host a website.

## 4.3 Dedicated hosting

In dedicated hosting, a customer uses an entire physical server.

Advantages:

- Greater control over hardware and configuration.
- Resources are not shared with unrelated hosting customers on that server.

Disadvantages:

- Higher cost.
- Requires more administration.

## 4.4 Cloud hosting

Cloud hosting uses cloud infrastructure to provide computing, storage, networking, and other resources.

Examples include:

- AWS EC2
- AWS Lightsail



- Azure Virtual Machines
- Google Cloud Compute Engine

In your AWS practical, an EC2 instance running RHEL and Apache is an example of a cloud-hosted web server.

## 5. What is a web hosting environment?

A web hosting environment consists of the components required to make a website accessible.

For a basic Linux web server, the main components are:

Client

A browser on a laptop or mobile device that requests the website.

DNS

Finds the IP address associated with the domain name.

Network and firewall

Allows the browser to reach the web server through the required network ports.

Web server (Apache)

Receives requests and serves website files or application responses.

Database (if required)

Stores information such as user accounts, posts, products, and orders.

A static website may only require a web server and its files. A dynamic website, such as WordPress, may also need PHP and a database.

### 5.1 Static website

A static website serves files that have already been created.

For example:

- HTML
- CSS

- JavaScript
- Images

When a user requests a static webpage, the web server sends the existing file to the browser.

Example: A company website containing only its name, contact information, and services.

## 5.2 Dynamic website

A dynamic website generates content based on the request or information stored in a database.

For example, when you log in to a shopping website, the server may retrieve your account details and orders from a database.

A common Linux hosting setup for a dynamic website is called a LAMP stack.

| Letter | Component        | Purpose                                            |
|--------|------------------|----------------------------------------------------|
| L      | Linux            | Operating system                                   |
| A      | Apache           | Web server                                         |
| M      | MySQL or MariaDB | Database                                           |
| P      | PHP              | Programming language used by many web applications |

WordPress is an example of a PHP-based application that commonly uses MySQL or MariaDB.

## 6. What is a domain name?

A domain name is a human-readable name used to access a website or other internet service.

Examples:

- google.com
- amazon.com
- myawshosting.net

Computers communicate using IP addresses, but people find domain names easier to remember.

A domain name is registered through a domain registrar, which is a company authorized to register domain names under the applicable domain registry rules.

Examples of domain registrars include GoDaddy, Namecheap, and others.

## Domain name vs website hosting

These are two different things.

| Domain name                            | Web hosting                                        |
|----------------------------------------|----------------------------------------------------|
| The name used to access a website.     | The server or service where the website is hosted. |
| Example: example.com                   | Example: An EC2 instance running Apache.           |
| Registered through a domain registrar. | Provided by a hosting company or cloud platform.   |

Buying a domain name does not automatically create a website.

Similarly, you can host a website on a server and access it using its IP address even if you have not purchased a domain name.

## 7. What is DNS?

DNS stands for Domain Name System.

DNS translates human-readable domain names into IP addresses that computers use to communicate.

In simple words, DNS is like the contacts list on your phone.



## Phone contacts

You search for a person's name, and your phone finds their number.



## DNS

You enter a website's name, and DNS helps find the server's IP address.

For example, when you enter [www.example.com](http://www.example.com), your computer needs the corresponding IP address to connect to the server.

DNS provides the answer through a system of DNS records and servers.

## 7.1 Why do we need DNS?

Imagine that you need to remember the IP address of every website you visit.

This would be difficult because:

- IP addresses are harder to remember than names.
- Websites may change their IP addresses.
- Large websites may use many IP addresses.
- Different services need different DNS records.

DNS allows people to use a consistent domain name while the underlying server addresses can change.

### Example

Suppose a website initially uses one server IP address:

`example.com → 192.0.2.10`

Later, the website moves to another server:

`example.com → 192.0.2.20`

The DNS record can be updated to point to the new address. Users can continue using the same domain name.

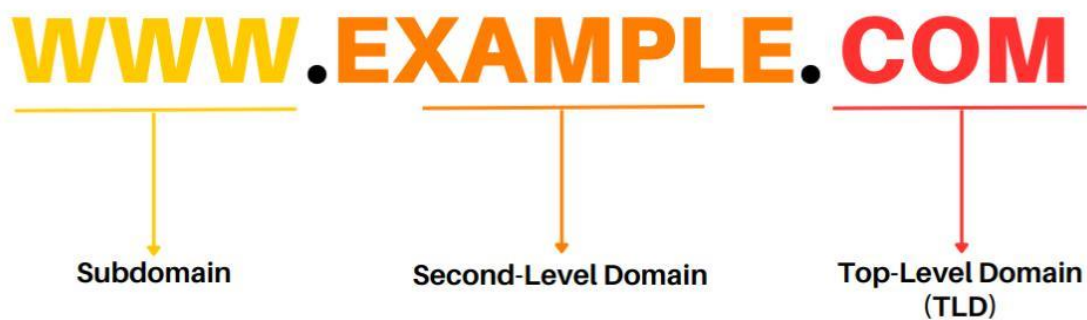
The IP addresses in this example are reserved for documentation.

## 8. Domain name structure

A domain name is organized into different levels.

Consider:

[www.example.com](http://www.example.com)



| Part    | Name                   | Explanation                                                          |
|---------|------------------------|----------------------------------------------------------------------|
| .       | Root                   | The top of the DNS hierarchy. Usually hidden in normal domain names. |
| com     | Top-Level Domain (TLD) | The domain extension.                                                |
| example | Second-Level Domain    | The registered name under .com.                                      |

|     |                       |                                |
|-----|-----------------------|--------------------------------|
| www | Subdomain or hostname | A name used under example.com. |
|-----|-----------------------|--------------------------------|

The complete DNS name is technically written with a final dot to represent the root:

[www.example.com](http://www.example.com).

Browsers and most DNS tools omit this final dot in everyday use.

## Common TLDs

| TLD  | Common use                                                                 |
|------|----------------------------------------------------------------------------|
| .com | Commercial and general-purpose websites                                    |
| .org | Commonly used by organizations                                             |
| .net | Originally intended for network-related organizations; now general-purpose |
| .in  | India country-code domain                                                  |
| .edu | Educational institutions in the United States under eligibility rules      |

A TLD does not necessarily tell you exactly what type of organization owns a domain.

# 9. How DNS works — complete explanation

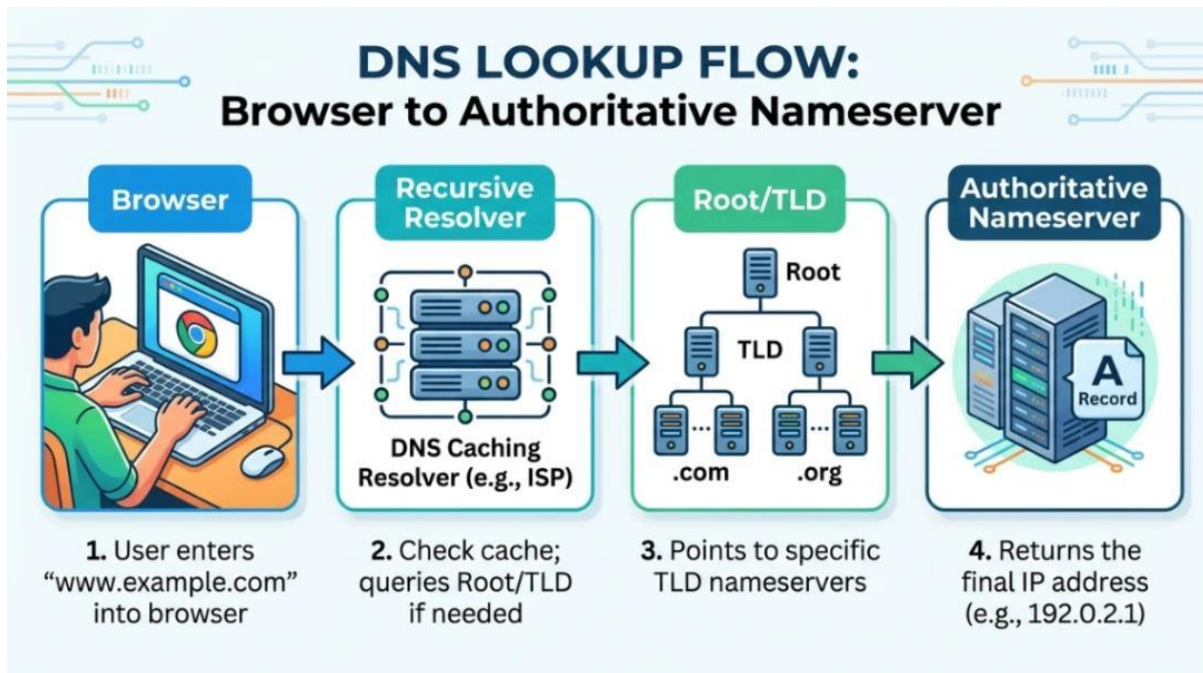
This is one of the most important concepts in web hosting.

Suppose you open your browser and enter:

[www.example.com](http://www.example.com)

Your computer does not automatically know the IP address of the website. It needs to perform a DNS lookup.

## DNS working diagram



## Step-by-step DNS resolution

1. Step 1 — User enters a domain name

You type [www.example.com](http://www.example.com) into your browser and press Enter.

The browser needs an IP address to establish a connection to the website server.

2. Step 2 — Check the DNS cache

Your computer or DNS resolver may already have the IP address stored in its cache.

If a valid cached answer exists, it can be returned without querying the DNS hierarchy again.

If the answer is not available, the request is sent to a DNS resolver.

3. Step 3 — Recursive DNS resolver

The recursive resolver is usually provided by your ISP or a public DNS service.

Examples include Google Public DNS (8.8.8.8) and Cloudflare DNS (1.1.1.1).

The resolver searches for the requested DNS record on behalf of your computer.

#### 4. Step 4 — Root DNS server

If the resolver has no cached answer, it queries a root DNS server.

The root server does not normally provide the IP address for [www.example.com](http://www.example.com).

Instead, it tells the resolver which nameservers are responsible for the .com TLD.

#### 5. Step 5 — TLD DNS server

The resolver queries the .com TLD server.

The TLD server provides a referral to the authoritative nameservers for example.com.

#### 6. Step 6 — Authoritative DNS server

The resolver queries the authoritative DNS server for the requested record.

This server contains the DNS records for the domain and returns the requested answer, such as the A record or AAAA record.

#### 7. Step 7 — IP address returned

The recursive resolver returns the answer to your computer and caches it according to its TTL.

The browser can now connect to the website server using the IP address.

#### 8. Step 8 — Website opens

The browser establishes the required connection and sends an HTTP or HTTPS request.

The web server responds with the requested webpage, and the browser displays it.

Important: The root, TLD, and authoritative servers are not necessarily contacted on every visit. Caching can allow a resolver to answer a query immediately.

## 10. Types of DNS servers

There are four main DNS components to understand.



## 10.1 DNS Resolver

A DNS resolver receives queries from your computer and finds the requested DNS information.

It can search its cache or contact other DNS servers to find the answer.

Example: Google Public DNS at 8.8.8.8.

Remember: The resolver does the searching on behalf of your computer.

## 10.2 Root DNS Server

The root DNS server is at the top of the DNS hierarchy.

It directs the resolver to the correct TLD nameservers.

For example, if the resolver is searching for `example.com`, the root server directs it to the `.com` nameservers.

There are 13 named root server identities, operated through many physical server instances around the world.

Remember: Root servers direct queries to TLD servers. They do not store every website's IP address.

## 10.3 TLD DNS Server

TLD stands for Top-Level Domain.

TLD servers manage information about domains registered under a particular extension, such as `.com`, `.org`, or `.in`.

For example, the `.com` TLD server directs the resolver to the authoritative nameservers for `example.com`.

Remember: TLD servers help the resolver find the correct authoritative nameservers.

## 10.4 Authoritative DNS Server

An authoritative DNS server contains the actual DNS records for a domain.

For example, the DNS zone for `example.com` may contain records such as:

- A record for the website's IPv4 address.
- MX records for email.
- TXT records for verification and email security.

When queried for a record it manages, the authoritative server provides the official answer.

Remember: The authoritative server provides the DNS records for the domain.

### Quick comparison

| DNS component        | Main job                              |
|----------------------|---------------------------------------|
| Resolver             | Finds the answer.                     |
| Root server          | Directs to the TLD.                   |
| TLD server           | Directs to authoritative nameservers. |
| Authoritative server | Provides the DNS records.             |

## 11. DNS records

DNS records are entries stored in DNS that provide information about a domain. Different record types perform different functions.

### 11.1 A record

An A record maps a domain name to an IPv4 address.

**Example:** `example.com A 192.0.2.10`

**Simple meaning:** Domain name to IPv4 address.

### 11.2 AAAA record

An AAAA record maps a domain name to an IPv6 address.

**Example:** example.com AAAA 2001:db8::10

**Simple meaning:** Domain name to IPv6 address.

### 11.3 CNAME record

A CNAME record makes one domain name an alias, or another name, for a different domain name.

**Example:** [www.example.com](http://www.example.com) CNAME example.com

**Simple meaning:** One domain name points to another domain name.

### 11.4 MX record

MX stands for Mail Exchange. An MX record specifies which mail servers receive email for a domain.

**Example:** example.com MX mail.example.com

**Simple meaning:** Specifies the mail servers for a domain.

### 11.5 NS record

NS stands for Name Server. An NS record identifies the authoritative nameservers responsible for a domain or DNS zone.

**Example:** example.com NS ns1.example.net

**Simple meaning:** Tells which DNS servers manage the domain's DNS records.

### 11.6 TXT record

A TXT record stores text information in DNS. It is commonly used for domain ownership verification, SPF information, and DKIM public-key information.

**Example:** example.com TXT "verification-code=12345"

**Simple meaning:** Stores text that other services can check for verification and security purposes.

## 11.7 SOA record

SOA stands for Start of Authority. It contains important information about a DNS zone, such as the primary nameserver, zone administrator, serial number, and timing values used by DNS servers.

**Example:** The SOA record for `example.com` contains the zone's serial and refresh settings.

**Simple meaning:** Provides key information and settings for a DNS zone.

## 11.8 PTR record

A PTR record is used for reverse DNS lookup. It maps an IP address to a domain name.

**Example:** An IP address can have a PTR record that returns `mail.example.com`.

**Simple meaning:** IP address to domain name.

## 11.9 SRV record

An SRV record specifies the hostname and port for a particular service, along with priority and weight information.

**Example:** A service record can identify the server and port used by a service.

**Simple meaning:** Specifies where a particular service is available.

## 11.10 CAA record

A CAA record specifies which certificate authorities are allowed to issue TLS/SSL certificates for a domain.

**Example:** A CAA record can authorize a particular certificate authority to issue certificates for `example.com`.

**Simple meaning:** Specifies which certificate authorities may issue certificates for the domain.

## DNS record revision table

| Record | Purpose              |
|--------|----------------------|
| A      | Maps a name to IPv4. |
| AAAA   | Maps a name to IPv6. |

|              |                                              |
|--------------|----------------------------------------------|
| <b>CNAME</b> | Makes one name an alias of another.          |
| <b>MX</b>    | Specifies mail servers.                      |
| <b>NS</b>    | Identifies authoritative nameservers.        |
| <b>TXT</b>   | Stores text for verification and security.   |
| <b>SOA</b>   | Provides DNS zone information and settings.  |
| <b>PTR</b>   | Maps an IP address to a name.                |
| <b>SRV</b>   | Specifies service hostname and port.         |
| <b>CAA</b>   | Specifies permitted certificate authorities. |

## 12. DNS caching and TTL

### What is DNS caching?

DNS caching means temporarily storing DNS answers so that the same information does not need to be looked up repeatedly.

For example:

1. You visit `example.com`.
2. The resolver finds the IP address.
3. The resolver stores the answer in its cache.
4. When you visit the same domain again, the cached answer may be returned directly.

This reduces DNS lookup time and the number of queries sent to other DNS servers.

### What is TTL?

TTL stands for Time To Live.

It specifies how long a DNS answer may remain in a cache before it needs to be refreshed.

For example:

```
example.com      A      192.0.2.10      TTL: 3600
```

A TTL of 3600 seconds is one hour.

After the cached answer expires, the resolver must obtain a fresh answer before relying on it as current DNS information.

## Why is TTL important?

Suppose a website moves from one server to another.

If the old DNS answer is still cached, some users may continue using the old IP address until their cached answer expires or is refreshed.

A shorter TTL can allow DNS changes to be picked up sooner, but it can also increase the number of DNS queries.

# 13. Domain registration and DNS hosting

These are related but different services.

## Domain registrar

A domain registrar is a company through which you register a domain name.

Examples include GoDaddy and Namecheap.

When you register a domain, you obtain the right to use that domain name for the registration period, subject to the applicable rules.

## DNS hosting provider

A DNS hosting provider operates DNS servers that store and answer queries for your domain's DNS records.

Examples include Amazon Route 53, Cloudflare DNS, and other DNS hosting services.

## How they work together

Suppose you register `mywebsite.com` through a registrar and host your DNS records with a DNS provider.

The process is:

1. Register `mywebsite.com`.
2. Configure the domain's nameservers at the registrar.
3. Create DNS records at the DNS hosting provider.
4. Configure an A record or another suitable record for the website.

5. Users can resolve the domain and connect to the hosting server.

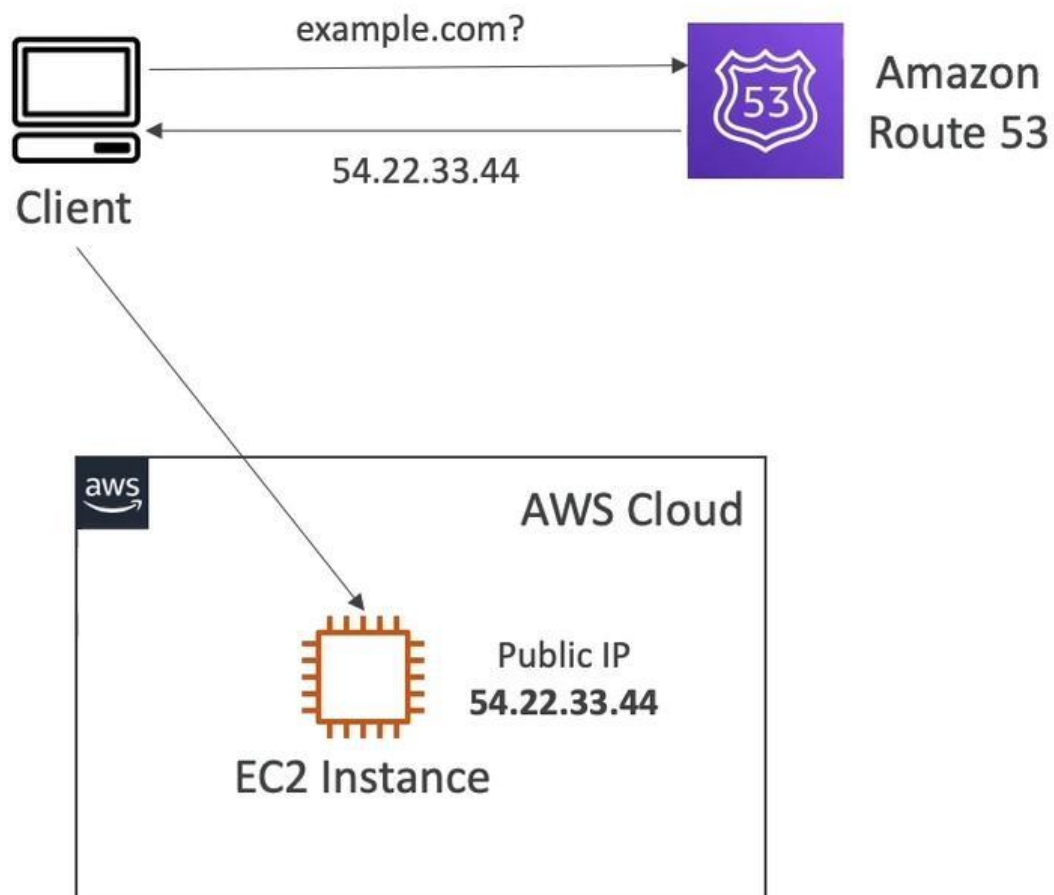
The registrar manages the domain registration. The DNS hosting provider manages the DNS records. The web hosting provider serves the actual website.

## 14. DNS and AWS web hosting

Let's connect DNS with the AWS practical you have been learning.

Suppose you have:

- Domain: myawshosting.net
- EC2 instance running RHEL 10.2.
- Apache installed on the EC2 instance.
- A public IP address assigned to the instance.



The basic setup works like this:

| Component                        | Purpose                                                                         |
|----------------------------------|---------------------------------------------------------------------------------|
| Domain name                      | The name users enter in their browser.                                          |
| Route 53 or another DNS provider | Stores the DNS records for the domain.                                          |
| DNS A record                     | Points the domain to the EC2 public IPv4 address, if using a direct IPv4 setup. |
| EC2 instance                     | Runs the Linux operating system and Apache.                                     |
| Apache                           | Serves the website files to users.                                              |

## Example

Suppose the DNS provider contains:

```
myawshosting.net    A    192.0.2.10
```

The IP address is illustrative and reserved for documentation.

When a user enters `myawshosting.net`, DNS returns the configured address. The browser then connects to the server, and Apache serves the website.

For a real AWS deployment, the instance must also have appropriate network connectivity, security group rules, and a suitable public IP or other reachable endpoint. A direct record pointing to an EC2 public IP can require updating if that address changes.

# 15. Common DNS problems

DNS problems can prevent users from reaching a website even when the web server itself is running.

| Problem | Simple explanation |
|---------|--------------------|
|---------|--------------------|



|                        |                                                                    |
|------------------------|--------------------------------------------------------------------|
| Incorrect A record     | The domain points to the wrong IPv4 address.                       |
| Incorrect nameservers  | The domain delegates DNS to the wrong DNS servers.                 |
| DNS server unavailable | The resolver or authoritative server does not respond.             |
| Cached old record      | A resolver still has an earlier DNS answer in its cache.           |
| Domain does not exist  | The requested name has no valid DNS record or domain registration. |

For example, if Apache is running correctly but the domain's A record points to the wrong IP address, visitors will not reach the intended web server.

## Basic DNS troubleshooting commands on Linux

These commands are useful for checking DNS configuration and resolving domain names.

### Check the IP address returned for a domain

```
dig example.com
```

### Check the A record

```
dig example.com A
```

### Check the nameservers

```
dig example.com NS
```

### Check using nslookup

```
nslookup example.com
```

dig and nslookup are DNS lookup tools. They query DNS servers and display the answers they receive.

## 16. Final revision — Web Hosting vs DNS

| Web hosting                                       | DNS                                                             |
|---------------------------------------------------|-----------------------------------------------------------------|
| Stores or serves website content.                 | Translates domain names into DNS answers, such as IP addresses. |
| Uses web server software such as Apache or Nginx. | Uses DNS resolvers and DNS servers.                             |
| Handles HTTP and HTTPS requests.                  | Handles DNS queries.                                            |
| Example: EC2 running Apache.                      | Example: Route 53 hosting an A record.                          |

### Conclusion

Web hosting is the process of storing and serving website content through a server. DNS is the naming system that helps users locate those servers using easy-to-remember domain names.

When a user opens a website, DNS helps find the server's address, the browser connects to that server, and the web server returns the requested content.

Understanding these concepts provides the foundation for learning Apache, Linux server administration, domain configuration, and AWS web hosting.